11.3.4 The must_not clause

The opposite of `must`, as you can guess, is the `must_not` query clause. For example, on a shopping site, we may want to search for products with specific details, asking the retailer to *ignore* products that are not available. Figure 11.3 shows an example (from a UK retailer, John Lewis).

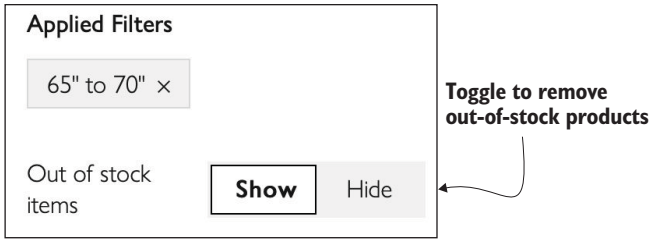


Figure 11.3 Hiding out-of-stock items during search

The search engine hides all out-of-stock items before showing the search results. This is the sort of functionality the `must_not` clause can satisfy.

Like the `must` clause, `must_not` accepts an array of leaf queries to build advanced search criteria. The query's sole aim is to filter out matches that do *not* meet the criteria specified. The best way to understand this is with an example. The query in the following listing searches for all TVs that are not a specific brand (in this case, Samsung or Philips).

Listing 11.5 Fetching TVs using `must_not` to exclude certain brands

```
GET products/_search
{
 "query": {
   "bool": {
     "must_not": [
       {
         "terms": {
           "brand.keyword": [
             "Samsung",
             "Philips"
           ]
         }
       }
     ]
   }
 }
}
```

A must_not clause to host a terms query

The terms query that searches for specific brands

This listing shows the `bool` query provided with a `must_not` clause that holds a `terms` query. At first, as expected, the `terms` query fetches products that are Samsung or Philips. As this `terms` query exists under a `must_not` clause, the effect is to negate what